

Timelock Audit



July 10, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Privileged Roles	7
Trust Assumptions	7
Notes & Additional Information	10
N-01 Style Guide Deviations	10
N-02 Incomplete and Inconsistent Abort Documentation	10
N-03 Missing Documentation	12
N-04 Incomplete Event Emitter Documentation	12
N-05 EPredecessorIsSelf Is Unreachable in Practice	13
N-06 is_operation_pending Excludes Cancellable Expired Operations	14
N-07 Configuration-Change Events Are Emitted on No-Op Updates	14
N-08 OperationCancelled Event Omits the Action Type	15
Conclusion	16
Appendix	18
Issue Classification	18

Summary

Type	Library	Total Issues	8 (8 resolved)
Timeline	From 2026-07-01 To 2026-07-03	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	8 (8 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [OpenZeppelin/contracts-sui](#) repository at commit [3619e3a](#).

In scope were the following files:

```
contracts
├── timelock
│   └── sources
│       └── timelock.move
```

System Overview

The `openzeppelin_timelock` package provides a hash-keyed operation timelock for Sui: a shared registry that enforces a mandatory, per-operation delay between proposing an action and executing it, giving stakeholders a reaction window before changes take effect. It is a Sui-native counterpart of OpenZeppelin's `TimelockController`. Because Move has no arbitrary-call dispatch, the timelock does not queue calldata; it stores each operation's typed parameters on-chain and, at execution time, hands them back through a one-shot ticket that the consuming module applies itself. The engagement covered a single module, `timelock`. Six example modules were additionally reviewed at a high level, as references for the intended integration patterns rather than as audited code: `example_amm`, `example_dual_governance`, and `example_upgrade_vault` under the timelock package's `examples` directory, and `example_reward_treasury`, `example_timelocked_treasury`, and `example_operator_handoff` under the access package's `examples` directory. All six are out of scope, and each is explicitly marked as not production ready.

The central object is a shared `Timelock` bound at creation to four role types (proposer, executor, canceller, and admin) drawn from the consumer's registry in `openzeppelin_access`, the timelock's only dependency. The timelock defines no roles of its own and never checks membership: each gated entry takes an `Auth<Role>` witness minted by the consumer's `AccessControl` and asserts only that `Role` matches the bound role type. Every operation is identified by a `deterministic keccak256 hash` over a domain tag and the operation's action type, parameter digest, predecessor, salt, and timelock id, so ids are reproducible off-chain and isolated per instance. An operation's ready window is fixed at schedule time, later configuration changes never move it, and execution is one-shot because the `Done` state is sticky.

The lifecycle of an operation proceeds through four flows:

- **Schedule.** A proposer calls `schedule`, passing the typed params by value together with an optional predecessor, a salt, and a delay no shorter than the configured minimum; the params are stored under the operation id, which the call returns.
- **Execute.** Once the delay has elapsed and before the grace window closes, an executor calls `execute`, or anyone calls `execute_open` when open-executor mode is enabled.

The operation is marked `Done` and an `ExecutionTicket` is minted: a no-ability hot potato carrying the stored params, which must be consumed in the same transaction.

- **Consume.** The consuming module redeems the ticket through `consume` by presenting a value of the operation's `Action` witness type, receiving the operation id and the typed params to apply directly.
- **Cancel.** A canceller drops a not-yet-executed operation and its stored params through `cancel`.

Two further mechanisms complete the interface. The capability-bound entries (`schedule_with`, `execute_with`, `execute_open_with`, and `cancel_with`) mirror the raw entries but additionally take an `OperationCap`, a stored, authority-free handle minted through `new_operation_cap` that pins one operation kind to one `Timelock` instance. The canonical-timelock check is therefore enforced structurally rather than hand-written by every consumer. The configuration values (minimum delay, grace period, and the open-executor flag) are managed by the timelock itself: each configuration change is a scheduled operation and is applied through an admin-gated, timelocked execute.

Security Model and Trust Assumptions

The timelock's own code enforces two properties: a scheduled operation cannot execute before its delay elapses or after its grace window closes, and every gated entry requires an `Auth<Role>` whose type matches the role bound at creation. Everything else, including which addresses hold the roles, whether the `Timelock` being called is the canonical one, and what an executed operation ultimately does, lives in the consumer's package and its `AccessControl` registry. An `Auth<Role>` cannot be forged or stockpiled: it is minted only by `new_auth` after the registry verifies that the role's home module matches and that the sender currently holds the role, and it is drop-only, so it never outlives its transaction. Ticket redemption is gated by witness types that only the consuming module can construct, and the atomicity of execute-and-apply follows from the ticket's lack of abilities. The trust assumptions below are by-design properties and integrator responsibilities rather than defects; an adopting protocol must account for them.

Privileged Roles

The system defines the following privileged roles:

- **Proposer:** schedules operations through `schedule` or `schedule_with`, committing the typed params, the ready window, an optional predecessor, and a salt.
- **Executor:** executes ready operations and mints their tickets through `execute` or `execute_with`. When the open-executor flag is on, this gate is lifted and `execute_open` / `execute_open_with` allow permissionless execution.
- **Canceller:** cancels any not-yet-executed operation through `cancel` or `cancel_with` by naming its stored `Params` type, dropping the params and returning the id to `Unset` so the same operation can be re-scheduled. Cancellation is the only path that removes records and frees ids.
- **Admin:** changes the configuration through the timelocked pairs `schedule_update_min_delay` / `execute_update_min_delay`, `schedule_update_grace_period` / `execute_update_grace_period`, and `schedule_set_open_executor` / `execute_set_open_executor`, with new values subject to the same bounds enforced at creation.

Trust Assumptions

The system relies on the following trust assumptions:

- The consumer's `AccessControl` registry governs membership of all four roles, so whoever administers that registry indirectly controls proposing, executing, cancelling, and configuration. The timelock checks only that the presented `Auth<Role>` matches the `role types bound at creation`, which are immutable for the object's lifetime. The registry's administration lifecycle is inherited as well: a default-admin renounce is `two-step and delay-gated`; once finalized, roles still administered by the root role can no longer be granted or revoked, and existing members keep their power until they renounce voluntarily.
- The role slots accept arbitrary types, and the constructor does not require them to be distinct. Binding one type to several slots merges the corresponding capabilities; reusing the proposer type in the admin slot, for example, lets every proposer administer the configuration. Distinct types should be bound unless shared authority is intended.
- A proposer controls every field of an operation: the params, the predecessor, the salt, and a delay that is floored by the configured minimum but `deliberately unbounded above`. Ids are allocated `first come, first served`, so a proposer can occupy an id another proposer intended to use, or park operations arbitrarily far in the future. Such

interference moves no value and never shortens a delay, is sidestepped by re-scheduling with a fresh salt, and is reclaimed by cancellation; the proposer role is trusted not to engage in it. The staging surface also covers configuration operations: the marker witness types are public, so a proposer can stage a configuration change through the generic `schedule`; the value takes effect only if an admin executes it, with the `bounds` [re-checked at apply time](#).

- The library does not enforce checks a consumer adds in its own scheduling wrapper: `schedule` is gated only by the proposer role and takes `Action` as a bare type parameter, so a proposer can always reach the library directly. Reliable enforcement points are the library's role gates and the consumer's apply logic after `consume`, so proposer-supplied parameters should be treated as untrusted until applied. Keeping `Params` construction package-private further prevents third parties from building schedulable values.
- Consumers must bind the canonical `Timelock`, as the library [cannot distinguish](#) a legitimate instance from an attacker-created one bound to the same role types. The `OperationCap` path makes the binding structural; consumers using the raw entries must assert the timelock id themselves.
- `Action` witness types must follow the documented [witness discipline](#): `drop`-only, one per consume function, and never constructible outside the consuming package, since any holder of a witness value can redeem the corresponding tickets.
- A predecessor being `Done` [does not imply its effect has been applied](#); consumers are responsible for ordering each execute, consume, and apply triple within a transaction. Predecessor wiring also carries a liveness responsibility: a dependent operation [cannot execute](#) while its predecessor is cancelled or expired, and it unblocks only when the predecessor is re-scheduled with byte-identical parameters (reproducing the same id) and then executed, or when the dependent itself is cancelled.
- A minimum delay of zero is [permitted by design](#): operations, including self-administered configuration changes, can then be scheduled with zero delay and become executable immediately, so the reaction window exists only if the deployer configures a nonzero minimum.
- When open-executor mode is enabled, [anyone can execute](#) a ready operation; safety rests on consumption remaining witness-gated, so a transaction that cannot consume the minted ticket aborts and reverts the state change.
- The schedule and execution windows are evaluated against the shared Sui `Clock`, which advances once per consensus commit; the system inherits its integrity and its commit-level granularity from the underlying network. At the extreme, a grace window configured shorter than the clock's update cadence, well below one second, may never be observed by any transaction; realistic configurations are unaffected.

- Operation ids embed [original package ids](#), so ids and in-flight operations survive consumer package upgrades: an operation scheduled before an upgrade executes against the post-upgrade apply logic, and scheduling a replacement never invalidates the original, which stays independently executable within its own window. Consumers that change apply semantics or supersede a scheduled operation are responsible for cancelling the outdated one.
- Operation records are never pruned automatically: a **Done** row [persists permanently](#) as replay protection (its stored params are removed at execution), while an expired operation keeps its row, its params, and its id until cancelled. Growth is bounded only by proposer and admin scheduling activity; entries are dynamic-field-backed, so the shared object itself does not grow, and the **store + drop** bound on params structurally keeps coins and other no-drop assets out of timelock storage. A long-lived timelock accepts unbounded state growth and the accrued storage deposits.

Notes & Additional Information

N-01 Style Guide Deviations

The module conforms to the style guide in most respects. The abort message strings, however, diverge from the [naming rule for error constants](#), which requires plain, user-facing English with no field, function, or type identifiers and no integer-type names:

- Five messages embed identifiers: `EWrongRole` names the `Auth` type, `EDelayTooShort` the `min_delay_ms` field, `EScheduleOverflow` the code expression `now + delay_ms` and the type name `u64`, and `EWrongParams` and `EWrongAction` the `Params` and `Action` type parameters. Each constant's `///` doc comment already carries the implementation detail, which is where the rule places it.
- The `EScheduleOverflow` message additionally misattributes the failure: it names only the `now + delay_ms` sum, but the error is raised both for [that sum](#) and for the [grace-period addition](#) `ready_at_ms + grace_period_ms`. The [constant's doc comment](#) describes both causes correctly; a frontend surfacing the raw message would misdiagnose the second one.

Consider rewording the five messages into identifier-free English and making the `EScheduleOverflow` message expression-agnostic so that it covers both overflow sites.

Update: Resolved in [pull request #438](#). The team stated:

| *Applied*

N-02 Incomplete and Inconsistent Abort Documentation

The `timelock` module documents each public function's error surface in an `#### Aborts` section. The [style guide](#) requires such sections to list every cause a function can raise,

including errors propagated from cross-module calls. Several lists diverge from the implementation:

- The `#### Aborts` sections of `execute_update_min_delay` and `execute_update_grace_period` omit `EInvalidConfig`, although both functions re-assert their configuration bounds at apply time (the `min-delay assert` and the `grace-period assert`). These asserts reject out-of-bounds values staged through the generic `schedule`, which performs no bound check, and both are exercised by `dedicated tests`. The sibling `schedule_update_min_delay` and `schedule_update_grace_period` docs do list the error.
- The `#### Aborts` list of `execute` names `EWrongAction` and `EWrongParams` as the second and third causes, although they are the `last two checks to fire` in `execute_internal`, after the `six operation-state checks`. The same list also merges those six causes into a single bullet without per-cause conditions, while the style guide asks for `one bullet per cause`.
- The `#### Aborts` section of `operation_params` attributes the `sui::dynamic_field` abort solely to a missing stored value ("Unset or already Done"). A third cause exists: the dynamic-field lookup key is derived from the parent object and the key (type and bytes), not from the value type, so for a pending operation the lookup succeeds and `df::borrow` then checks the stored value's type, aborting with `EFieldTypeMismatch` when the caller supplies a `Params` type different from the one the operation was scheduled with.
- Scheduling stores each new operation through `Table::add` and `df::add`. Both framework functions abort with `sui::dynamic_field::EFieldAlreadyExists` when the key already exists. In this module that case never occurs, because the `id-uniqueness check` aborts first with the module's own `EOperationAlreadyExists`. The style guide nonetheless `requires guarded cross-module aborts to be listed and marked as unreachable`, as part of the function's full error surface; the docs of `schedule`, `schedule_with`, the two `schedule_update_*` functions, and `schedule_set_open_executor` omit them.

Consider completing the affected `#### Aborts` sections by adding an `EInvalidConfig` bullet to both configuration executes, naming the type-mismatch cause on `operation_params`, and listing the guarded duplicate-key aborts marked as unreachable. Consider also reordering the `#### Aborts` list of `execute` to match evaluation order and splitting its merged bullet into one bullet per cause.

Update: Resolved in [pull request #439](#). The team stated:

| *Applied*

N-03 Missing Documentation

The `timelock` documentation omits information that integrators rely on:

- The `view-helpers block` is the only stretch of undocumented public functions in the repository: none of its fourteen functions carries a doc comment, from the one-line config getters through the six state views (`is_operation`, `is_operation_pending`, `is_operation_ready`, `is_operation_expired`, `is_operation_done`, and `operation_state`), while every other public function across the library's packages is documented, including trivial getters, and the adjacent `operation_params` is fully documented. Not every boundary is evident from the names either: `is_operation` returns `true` for an already-executed operation, since `Done` records persist.
- The operation-id preimage format is only partially specified. The `module header` and the `README` give the hash formula and the `IdInput` field order, but neither documents the `DOMAIN_TAG` byte string (whose `_1` segment reads as an unexplained format version), the `with_original_ids` `TypeName` resolution, or the BCS encoding of `TypeName`, so reproducing ids off-chain still requires reading the source. The `README` calls the `TypeName` encoding "the sharp edge for tooling" without specifying it, and while `MAX_DELAY_MS` is exposed through `max_delay_ms`, `DOMAIN_TAG` has no getter.

Consider adding a one-line summary to each view helper, and completing the id-derivation specification with the domain tag value, its version segment, and the `TypeName` resolution and encoding, optionally exposing `DOMAIN_TAG` through a getter for parity with `max_delay_ms`.

Update: Resolved in [pull request #440](#). The team stated:

| *Applied*

N-04 Incomplete Event Emitter Documentation

The `timelock` module documents each event's emitters in its doc-comment summary: `OperationScheduled` says "Emitted by `schedule`", and `OperationExecuted` says "Emitted by `execute` / `execute_open`". Both events are actually emitted inside shared internal functions (`schedule_internal` and `execute_internal`) that the self-administered configuration entries also call. The two `schedule_update_*` functions and `schedule_set_open_executor` therefore emit `OperationScheduled`, and the two `execute_update_*` functions and `execute_set_open_executor` emit `OperationExecuted` alongside the corresponding `*Changed` event, in one call. The

[ExecutionTicket documentation](#) shows the same gap in its mint clause ("Minted only by `execute` / `execute_open`"), even though its destroy clause already mentions the in-module self-admin path.

Off-chain consumers typically decide what to index from these summaries. A consumer reading them literally would attribute every `OperationScheduled` and `OperationExecuted` to the generic pipeline, and would not expect a configuration execution to produce an `OperationExecuted` together with a `*Changed` event for the same action.

Consider widening the two event summaries and the ticket's mint clause to name the self-administered emitters.

Update: Resolved in [pull request #442](#). The team stated:

| *Applied*

N-05 `EPredecessorIsSelf` Is Unreachable in Practice

`schedule_internal` derives the `operation_id` as a keccak256 hash over a preimage that includes the predecessor (computed by `hash_operation`) and then `asserts` `predecessor != id` with `EPredecessorIsSelf`. Since the compared id is a hash of a preimage containing the predecessor itself, triggering the abort would require the hash's output to equal a value contained in its own input: a 32-byte `predecessor` such that hashing a preimage embedding that exact value yields the value back. Keccak256 output is effectively uniformly random, so each candidate input succeeds with probability one in 2^{256} and no known property of the function shortens the search, making the condition unreachable in practice. The check is defense in depth.

The abort is nonetheless presented as an ordinary failure: it is listed without qualification in the `#### Aborts` sections of `schedule`, `schedule_update_min_delay`, `schedule_update_grace_period`, and `schedule_set_open_executor`, and `EPredecessorIsSelf` is the only one of the module's seventeen error constants without an `expected_failure` test, an expected consequence of the infeasibility. For guarded propagated aborts, the [style guide](#) keeps unreachable causes listed but marked as unreachable; the same treatment fits this directly raised, practically unreachable abort.

Consider marking the abort as unreachable in practice in the four `#### Aborts` bullets and in the constant's doc comment, rather than presenting it as an ordinary reachable failure.

Adding a test is not advisable, as constructing a triggering input would require solving the same self-referential search.

Update: Resolved in [pull request #441](#). The team stated:

| *Applied*

N-06 `is_operation_pending` Excludes Cancellable Expired Operations

The `is_operation_pending` view returns `true` only for the `Waiting` and `Ready` states and `false` once an operation is `Expired`, and it carries no doc comment stating this exclusion. Expired operations are not inert: `cancel` explicitly accepts `Waiting`, `Ready`, and `Expired` operations, the [README describes cancellation the same way](#), and an expired operation keeps its stored params and its `timestamps` entry until cancelled. The name also mirrors OpenZeppelin's `TimelockController`, where operations do not expire and `isOperationPending` covers every scheduled-but-unexecuted operation, so integrators porting assumptions from Solidity would expect the wider semantics.

A consumer that gates cancellation or cleanup on `is_operation_pending` therefore silently skips exactly the operations most in need of cleanup: expired operations return `false`, are never picked up by the gated path, and their stored params remain resident indefinitely.

Consider adding a doc comment to `is_operation_pending` stating that `Expired`, `Done`, and `Unset` operations return `false`, and directing consumers that gate cancellation or cleanup to `is_operation_expired` or to `is_operation` combined with `is_operation_done`.

Update: Resolved in [pull request #443](#). The team stated:

| *Applied*

N-07 Configuration-Change Events Are Emitted on No-Op Updates

The three configuration events are documented as firing on actual changes:

`MinDelayChanged` "when the configured `min_delay_ms` changes",
`GracePeriodChanged` "when the configured `grace_period_ms` changes", and
`OpenExecutorChanged` "when open-executor mode is toggled". The emissions are

unconditional: `execute_update_min_delay`, `execute_update_grace_period`, and `execute_set_open_executor` emit on every successful execution without comparing the previous and new values, so executing a configuration operation that carries the currently configured value emits an event whose previous and new fields are equal. An off-chain consumer relying on the documented wording would count such events as real changes or toggles.

Consider either guarding each emission behind an inequality check or rewording the three doc comments to state that the event fires whenever an update is applied, including a no-op update, so that consumers can filter out events whose previous and new values match.

Update: Resolved in [pull request #444](#). The team stated:

| *Applied*

N-08 `OperationCancelled` Event Omits the Action Type

The lifecycle events identify operations by kind: `OperationScheduled` and `OperationExecuted` both carry an `action: TypeName` field alongside the operation id. `OperationCancelled` records only the id and the canceller, even though the action is available at the emit site: `cancel_internal` removes the `OpTimestamp::Pending` record, whose `action` field is [discarded with it](#) immediately before the [event is emitted](#). An indexer tracking operation lifecycles by action type must therefore join every cancellation against the earlier `OperationScheduled` event by id to learn what was cancelled, and a per-action event stream cannot include cancellations directly.

Consider adding an `action: TypeName` field to `OperationCancelled`, populated from the stored `Pending` record before its removal, so that the cancellation event is self-describing and consistent with its schedule and execute counterparts. Alternatively, consider documenting the omission on the event.

Update: Resolved in [pull request #445](#). The team stated:

| *Applied*

Conclusion

The `openzeppelin_timelock` package provides a hash-keyed operation timelock for Sui, a Sui-native counterpart of OpenZeppelin's `TimelockController` that stores each operation's typed parameters on-chain and, after a per-operation delay, releases them through a one-shot hot-potato ticket that the consuming module applies itself. OpenZeppelin audited the single `timelock` module, assessing the operation state machine and its window enforcement together with the deterministic derivation of operation ids. The assessment also covered the binding of the four role types to the consumer's `AccessControl` registry, the role-gated minting and witness-gated consumption of execution tickets, the capability-bound entries that pin operations to a canonical timelock, and the self-administered configuration pipeline. Six example modules under the timelock and access packages were outside the scope of this engagement and were reviewed only at a high level as references for the intended integration patterns.

No issues of critical, high, medium, or low severity were identified within the audited scope. Eight notes were raised, concentrated in three areas: the precision and completeness of the documentation, conformance with the library's own conventions, and the observability the event schema offers to off-chain consumers. Examples include the `is_operation_pending` view returning `false` for expired operations that remain cancellable, configuration-change events emitted even when an update re-applies the already-configured value, and the `OperationCancelled` event omitting the action type that indexers need to classify a cancellation without joining against earlier events. The core guarantees (that no operation executes before its delay elapses or after its grace window closes, that execution happens at most once, and that ticket consumption stays gated by the consumer's witness types) were not found to be affected.

The codebase is cleanly structured as a single module with extensive documentation and is accompanied by a test suite exercising the state machine, the role gates, and nearly the entire error surface through expected-failure tests. Because the timelock delegates role membership to the consumer's registry and leaves canonical-instance binding, witness discipline, and apply-time validation to the integrator, its effective security rests on correct integration, as set out in the trust assumptions. Addressing the surfaced items would tighten the documentation and the event schema, reducing the chance that an integrator or indexer adopts the module with a weaker model than intended.

The OpenZeppelin development team is thanked for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.